PLB
FORMATION INFORMATIQUE ET MANAGEMENT

Les nouveautés Spring5

David THIBAU – 2018

david.thibau@gmail.com

Agenda

- Introduction
  - Nouveautés Spring5
  - Java8 et Java9
  - Nouveautés JUnit5
  - Programmation réactive
- Spring Reactor
  - Présentation, dépendances, mise en place
  - Cœur de l'API : Mono et Flux
  - Threads, Scheduler
  - Gestion des erreurs, debugging, Tracing
- Persistance et Web
  - SpringData
  - Spring WebFlux
  - Client réactif
  - Reactive websockets
  - Sécurité
- Tests et Spring Cœur
  - Spring cœur et nouvelles annotations
  - Annotations JUnit
  - Test des clients web

Introduction

Nouveautés Spring 5
Java8 et Java9
JUnit5
Programmation Réactive

Nouveautés Spring 5

- Java 8+
- Programmation réactive avec *Spring WebFlux*
- Support avec Kotlin 1.1+
- Compatibilité avec les extensions de JUnit 5
- JPA 2.1 et Hibernate 5
- Abandon du support pour de nombreuses librairies : *PortletMVC*, *JDO*, *Guava caching*, *JasperReports*, *OpenJPA*, *Tiles 2*, *XMLBeans*, *Velocity*.
- Java EE 8 API level (e.g. Servlet 4.0 et HTTP/2)
- Compatible Spring Boot 2.x

Java 8 et 9

Dans sa version 4, Spring était compatible avec Java 6. Avec la version 5, le code du framework a été réécrit en Java 8.

Les développeurs en ont donc profité pour :

- Améliorer l'accès aux paramètres d'une méthode en utilisant les améliorations de la réflexivité de Java 8
- Utiliser les **méthodes par défaut** des interfaces
- Les **lambdas**, qui sont désormais utilisés dans les API Spring
- Et dans une optique de compatibilité avec Java 9, l'instanciation des classes se fera désormais par Constructor#newInstance au lieu de Class#newInstance qui va devenir obsolète.

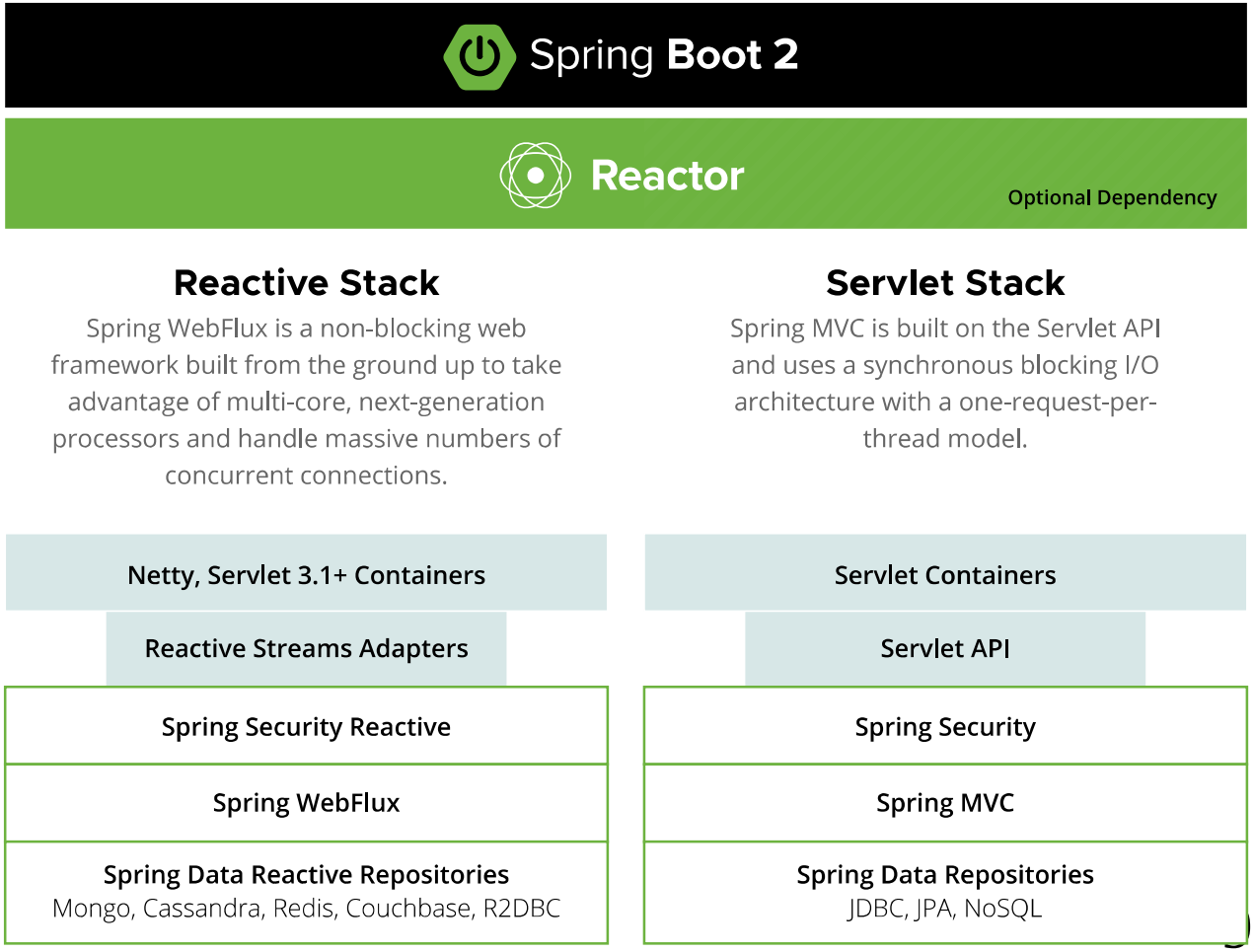
Programmation réactive

Spring se met à la programmation réactive avec ***WebFlux*** dans sa version 5.

Concrètement, il s'agit de pouvoir implémenter une application web (API REST) ou des clients HTTP de manière réactive.

Pour ce faire, Spring 5 intègre désormais ***Reactor*** et permet de manipuler les objets ***Mono*** (1 objet) et ***Flux*** (N objet(s)).

Stack Spring



Rappels Java 8 / Java 9

Lambda expressions

Les ***expressions Lambda*** (ou closure ou anonymous function) sont un moyen de simplifier le code

Exemple:

```
button.addActionListener(new ActionListener() {
@Override
public void actionPerformed(ActionEvent e) {
  operationQuelconque(e);
}
});
```

Équivalent à:

```
button.addActionListener( e ->operationQuelconque(e) );
```

L'expression Lambda s’écrit entre les parenthèses d'appel à une méthode

La méthode respecte une interface fonctionnelle (*Functional interface*)

Utilisation

```
// Appel de méthodes
Arrays.sort(testStrings,(s1,s2)→s1.length()-s2.length())
// Variables
AutoCloseable c = ()→cleanupForTryWithResources() ;
// Retour implicite
(arg1,arg2) -> qqch;
// Parenthèses omises si 1 argument
button1.addActionListener(
  event->setBackground(Color.BLUE));
// Accès en lecture à une variable locale
Color b2c = Color.GREEN;
boutton2.addActionListener(event -> setBackground(b2c));
```

Référence de méthode

Les **références de méthode** sont utilisées lorsque la lambda expression ne fait qu'appeler une autre méthode.

```
// Référence à méthode statique
String::valueOf
// Méthode d'instance sur un objet
s::toString
// Méthode d'instance non liée à un objet particulier
String::toUpperCase
// Constructeur
String::new
```

Interface Fonctionnelle

Le package ***java.util.function*** contient plus de 40 interfaces fonctionnelles destinées à faciliter l'écriture d'expressions Lambdas ou de références de méthodes

Par exemple :

Consumer : Fonction sans retour acceptant un type *T* en argument

Supplier : Fonction sans argument retournant un type *T*

Predicate : Test d’une condition

Function : Fonction prenant un type *T* en argument et retournant un type *R*

Stream : Introduction

Un ***Stream*** permet d'envelopper temporairement des données issues de tableaux ou de collections afin de les traiter massivement d'une manière efficace

- il permet d'effectuer des opérations séquentielles ou en parallèle sur les données
- Adapté aux expressions Lambda
- Pas d’accès indexé aux données

Un *Stream* est consommable, et de ce fait il n’est pas possible de créer une référence sur un *stream* pour un usage ultérieur

Types d’opérations

- Une opération **intermédiaire** retourne un nouveau Stream
  - *filter* et *map* par exemple
- Une opération **terminale** retourne un résultat ou produit un effet de bord
  - *forEach*, *toArray*, *min*, *max*, *findFirst*, *anyMatch*, *allMatch* etc...
- Une opération **intermédiaire est appelée court-circuit** si elle produit un Stream fini à partir d’un Stream infini
  - exemple *limit()* et *skip()*
- Une opération **terminale est appelée court-circuit** si elle peut terminer une opération dans un temps fini sur un stream infini
  - exemple *anyMatch, allMatch, noneMatch, findFirst* et *findAny*

Exemples

```
// forEach
Stream<Employe> employes = getEmployes().stream();
employes.forEach(e -> e.setSalaire(e.getSalaire() * 11/10)) ;
// map
Double[] carres = Stream.of(valeurs).map(n ->
                                        n * n).toArray(Double[]::new);
// filter
Integer[] evens = Stream.of(nums).filter(n ->
                                      n%2 == 0).toArray(Integer[]::new);
// findFirst
Stream<String> amis = Stream.of("Nicolas","Sophie","Chloe");
Optional<String> prenoms = amis.filter(i ->
                                         i.startsWith("S")).findFirst();
// Reduce
List<Double> nums = Arrays.asList(1.2, -2.3, 4.5, -5.6);
double max = nums.stream().reduce(Double.MIN_VALUE, Double::max);
double produit = nums.stream().reduce(1, (n1, n2) -> n1 * n2);
```

CompletableFuture

CompletableFuture améliore *Future* avec les lambda expression, la gestion des erreurs, la possibilité de chaîner les traitements asynchrones, ou de les exécuter en //

```
CompletableFuture<String> completableFuture
  = CompletableFuture.supplyAsync(() -> "Hello");

CompletableFuture<String> future = completableFuture
  .thenApply(s -> s + " World");

assertEquals("Hello World", future.get());
```

Modules Java9

Similaire à OSGi : Les **modules** ont des dépendances et peuvent exporter une API publique et garder les détails d'implémentation cachés/privés

Pas obliger d'avoir tout Java pour exécuter des programmes
=> Permet d'exécuter Java sur de plus petit device

Autres nouveautés Java9

- Un nouveau client HTTP supportant HTTP/2 et les WebSocket
- API Process pour contrôler les processus natifs
- Framework *Publish/Subscribe* similaire à *ReactiveStream*
- Logging de la JVM (GC, Threads, Compiler, Memory, ...)
- *Immutable Set*, *Optional Stream*

JUnit 5

Le framework *JUnit5*

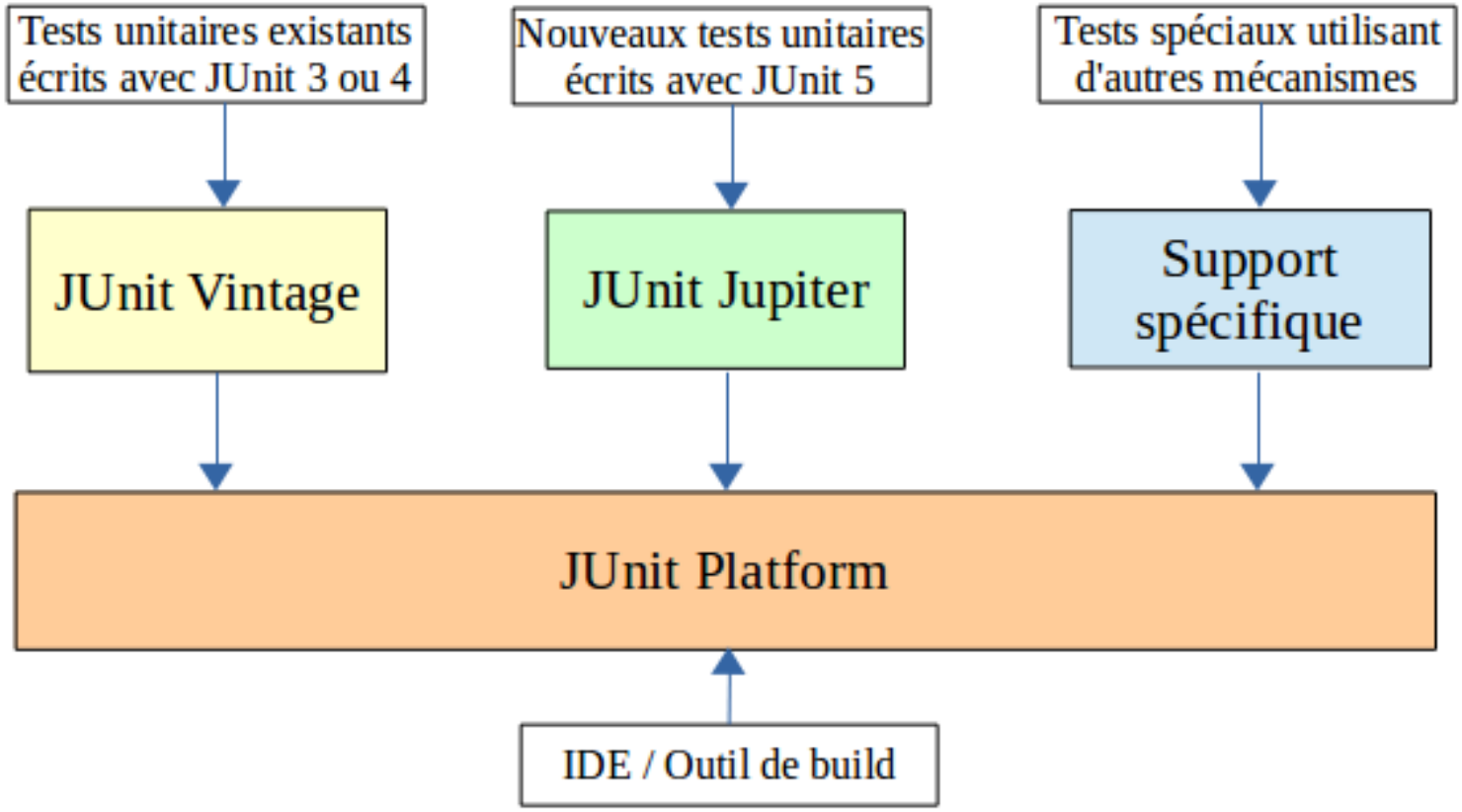
Avec *JUnit* 5 , *JUnit* devient compatible Java8 et Lambda expression

JUnit 5 est composé de 3 sous-projets :

- ***JUnit Platform*** : Fondation pour lancer les tests.
  Définit l'API *TestEngine.* Intégration avec outils de build.
  Permet d'exécuter des tests *JUnit4*
- ***JUnit Jupiter*** : Permet l'exécution de tests JUnit5
- ***JUnit Vintage*** : Permet l'exécution de tests JUnit3 et 4

Projet exemples :
https://github.com/junit-team/junit5-samples

Compatibilité descendante



http://blog.ippon.fr/2017/05/15/junit-5/

Annotations JUnit5 After,Before

@BeforeAll, ***@AfterAll*** : Méthodes exécutées une fois avant/après toutes les méthodes de Test.
Équivalent à *@BeforeClass*, *@AfterClass* de JUnit4

@BeforeEach, ***@AfterEach*** : Méthodes exécutées avant/après chaque méthode de test.
Équivalent à *@Before* et *@After* de JUnit4

Injection de dépendance

Afin que les méthodes de test soient exécutées en isolation, *JUnit* créée une nouvelle instance de la classe de test avant l'exécution de chaque méthode de test.

Jusqu'a *JUnit* 5, les constructeurs de classe de test ne pouvaient pas prendre d'arguments

Avec Junit 5, on peut profiter **d'injection de dépendance** lors des constructeurs ou appel de méthodes de test

Exemple

```
@DisplayName("TestInfo Demo")
class TestInfoDemo {

    TestInfoDemo(TestInfo testInfo) {
        assertEquals("TestInfo Demo", testInfo.getDisplayName());
    }

    @BeforeEach
    void init(TestInfo testInfo) {
        String displayName = testInfo.getDisplayName();
        assertTrue(displayName.equals("TEST 1") || displayName.equals("test2()"));
    }

    @Test
    @DisplayName("TEST 1")
    @Tag("my-tag")
    void test1(TestInfo testInfo) {
        assertEquals("TEST 1", testInfo.getDisplayName());
        assertTrue(testInfo.getTags().contains("my-tag"));
    }

}
```

Assertions

Assert est remplacée par *Assertions* qui ajoute 4 nouvelles méthodes :

- ***assertAll*** qui regroupe en argument des lambdas exécutant d'autres assertions
- ***assertThrows*** pour indiquer qu'on s'attend à voir survenir une exception
- ***assertTimeout*** ou ***assertTimeoutPreemptively*** selon que l'on souhaite attendre ou non la fin d'exécution d'un traitement testé par rapport à une contrainte de temps

Les messages optionnels d'échec sont en dernier paramètre (lazy initialisation)

Hamcrest n'est plus inclus.

Exemples *JUnit5*

```
assertAll("address",
() -> assertEquals("John", address.getFirstName()),
() -> assertEquals("User", address.getLastName()) );

Throwable exception =
 assertThrows(IllegalArgumentException.class, () ->
 {
 throw new IllegalArgumentException("a message");
 });

assertTimeoutPreemptively(ofMillis(10), () -> {
Thread.sleep(100); });
```

Modèle d’extension JUnit5

Le modèle d’extension de JUnit5 repose dorénavant sur l’interface ***Extension*** avec l’annotation ***@ExtendWith***

Le moteur *JUnit* enregistre les *Extensions* présentes dans le classpath et les applique lorsqu’il voit une annotation *@ExtendWith* sur une classe ou sur une méthode.

Sous-classes d’extension

L’interface *Extension* est une classe marqueur. Ses sous-classes sont :

- *BeforeAllCallback*
- *BeforeEachCallback*
- *BeforeTestExecutionCallback*
- *TestExecutionExceptionHandler*
- *AfterTestExecutionCallback*
- *AfterEachCallback*
- *AfterAllCallback*

Toutes les méthodes définies reçoivent en argument une classe de contexte englobant les informations nécessaires.

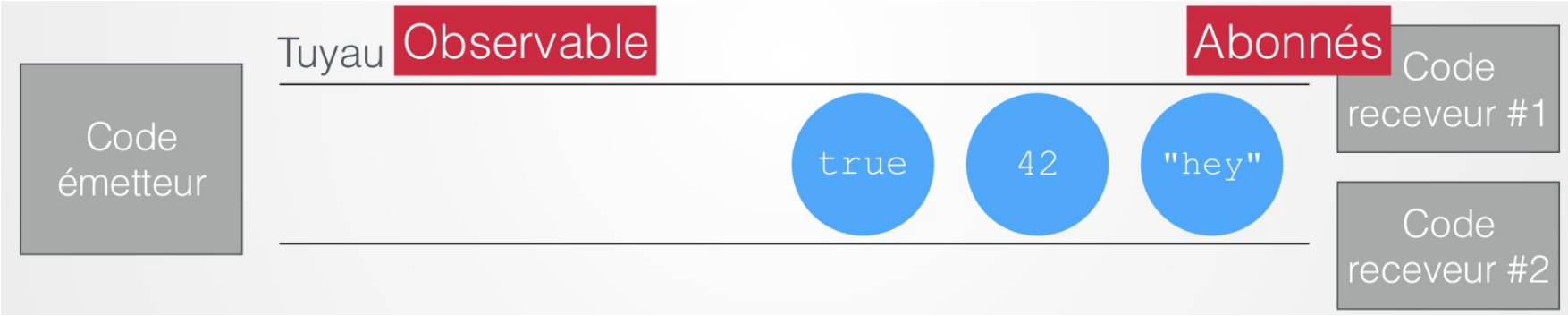
Programmation Réactive

Introduction

Consiste à construire son code à partir de flux de données : ***stream***.

Certaines parties du code émettent des données : les ***Observables***

D'autres réagissent : les ***Subscribers*** :



Le code émetteur peut envoyer un nombre illimité de valeurs dans le tuyau.

Le tuyau peut avoir un nombre illimité d'abonnés. Le tuyau est actif tant qu'il a au moins un abonné.

Les tuyaux sont des flux en temps réel : dès qu'une valeur est poussée dans un tuyau, les abonnés reçoivent la valeur et peuvent réagir.

Pattern et *ReactiveX*

La programmation réactive se base sur le pattern ***Observable*** qui est une combinaison des patterns *Observer* et *Iterator*

Elle utilise en plus la programmation fonctionnelle permettant de définir facilement des opérateurs

Elle est formalisée par l'API ***ReactiveX*** et de nombreuses implémentations existent pour différent langages (*RxJS*, *RxJava*, *Rx.NET*)

Combiner des transformations

Les données circulant dans le tuyau sont transformées grâce à une série d'opérations successives (aka "opérateurs") :

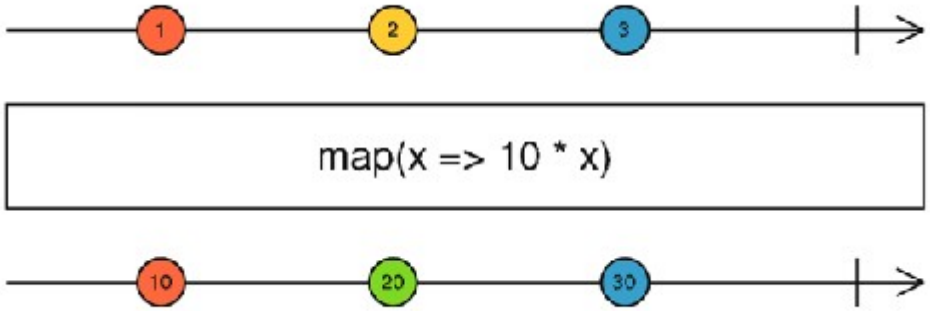
Les opérateurs à appliquer aux données sont déclarés une bonne fois pour toutes.

Ce fonctionnement déclaratif est pratique à utiliser et à débogguer.

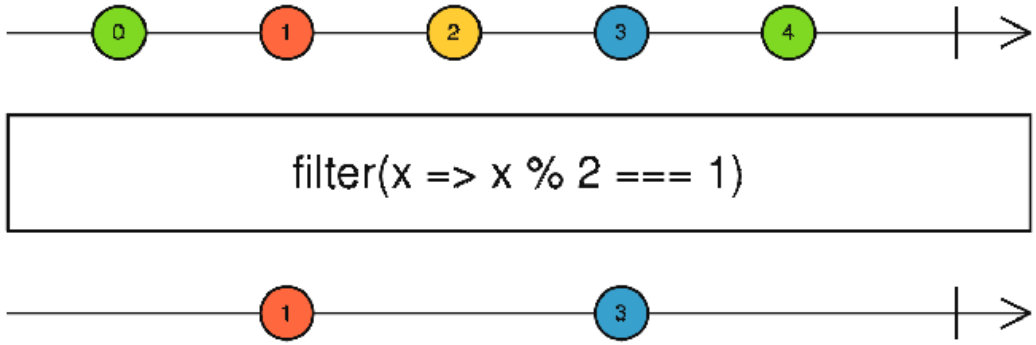
Marble diagrams

Pour expliquer les opérateurs, la doc utilise des ***marble diagrams***

Exemple *map* :



Exemple *filter*



Reactive Streams

Reactive Streams a pour but de définir un standard pour le traitement asynchrone de flux d'événements offrant une fonctionnalité de ***non-blocking back pressure***

Il concerne les environnements Java et Javascript ainsi que les protocoles réseau

Le standard permet l'inter-opérabilité mais reste très bas-niveau

Il correspond aux interfaces ***java.util.concurrent.Flow*** du JDK9

Interfaces Reactive Streams

```
public interface Publisher<T> {
    public void subscribe(Subscriber<? super T> s);
}
```

```
public interface Subscription {
    public void request(long n);
    public void cancel();
}
```

```
public interface Subscriber<T> {
    public void onSubscribe(Subscription s);
    public void onNext(T t);
    public void onError(Throwable t);
    public void onComplete();
}
```

```
public interface Processor<T, R> extends Subscriber<T>, Publisher<R> {
}
```

Back pressure

La notion de ***back pressure*** décrit la possibilité des abonnés de contrôler la cadence d'émission des événements du service qui publie.

- *Reactive Stream* permet d'établir le mécanisme et de fixer les limites de cadence.
- Si *l'Observable* ne peut pas ralentir, il doit prendre la décision de bufferiser, supprimer ou tomber en erreur.

Reactor

Reactor se concentre sur la programmation réactive côté serveur.

Il est développé conjointement avec Spring.

- Il fournit principalement les types de plus haut niveau ***Mono*** et ***Flux*** représentant des séquences d'événements
- Il offre un ensemble d'opérateurs alignés sur *ReactiveX*.
- C'est une implémentation de *Reactive Streams*

Spring Webflux

Spring WebFlux nécessite *Reactor* comme dépendance mais peut inter-opérer avec d'autres librairies via *Reactive Streams*.

Il se concentre donc sur des services web back-end réactifs.

C'est une alternative à *Spring MVC*

Ecosystème réactif

Spring Framework 5

Spring Boot 2

Spring Data 2

Spring Security 5

Spring Integration 5

Spring Cloud

Spring Reactor

Présentation
Cœur de l’API
Threads et Scheduler
Erreurs, Debug, Tracing

Introduction

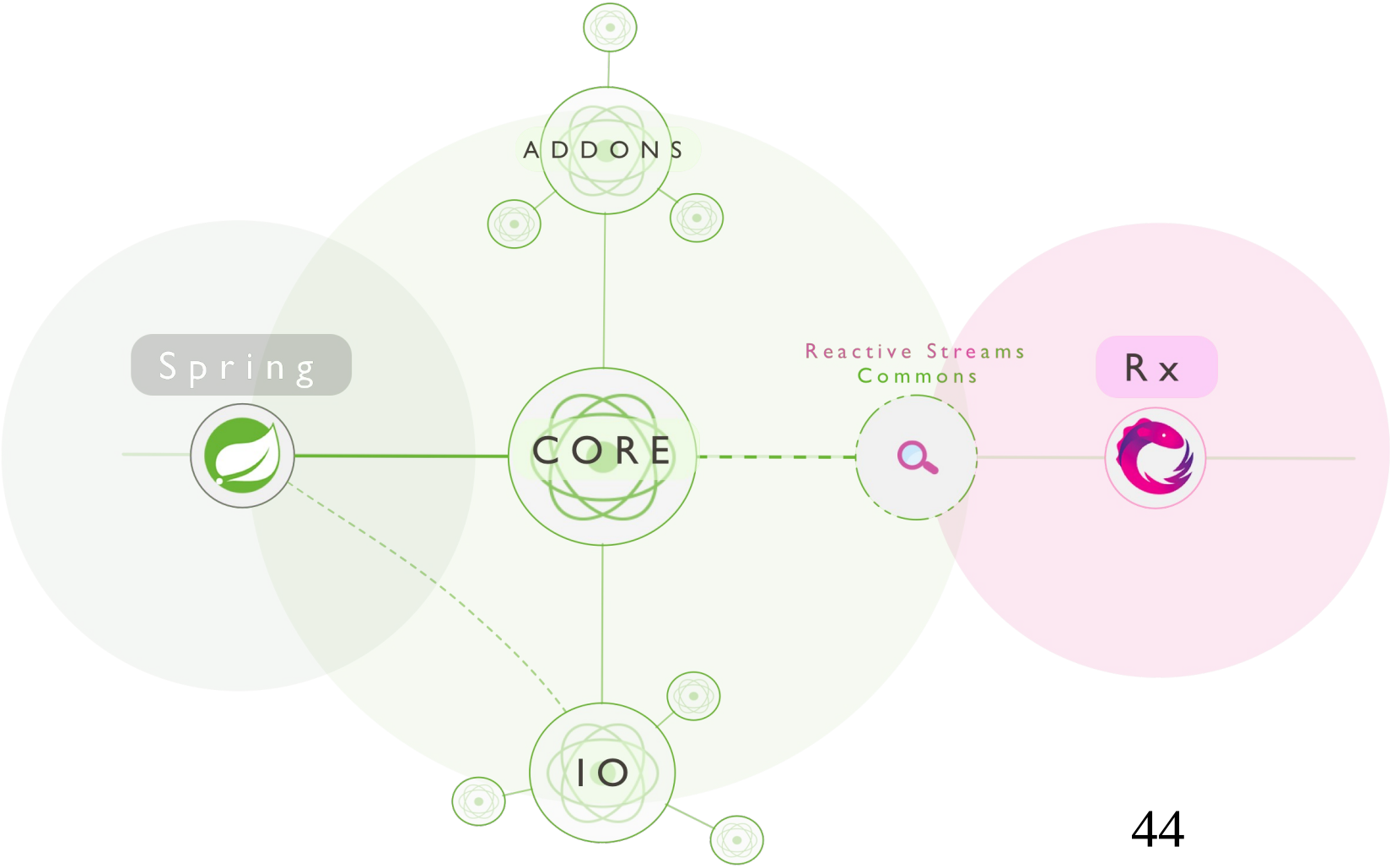
Spring Reactor est basé sur le pattern *reactive*

Son objectif est de construire des applications réactives complètement asynchrones

=> Traitement asynchrone d'importants volumes de requêtes provenant de 1 ou plusieurs producteurs

Par défaut, le bus d'événements n'est pas persistant mais implémenté en mémoire (comme *Spring Event Bus*)

Organisation projet



Dépendance Maven

```
<dependency>
    <groupId>io.projectreactor</groupId>
    <artifactId>reactor-core</artifactId>
    <version>${version}</version>
</dependency>
```

Cœur de l’API : Mono et Flux

2 Types

Reactor offre principalement 2 types Java :

- ***Mono*** : Flux de 0..1 éléments
- ***Flux*** : Flux de 0..N éléments

Tous les 2 sont des implémentations de l'interface ***Publisher*** de *Reactive Stream* qui définit 1 méthode :

```
void subscribe(Subscriber<? super T> s)
```

Le flux commence à émettre seulement si il y a un abonné

En fonction du nombre possible d'événements publiés, ils offrent des opérateurs différents

Flux

Un ***Flux<T>*** représente une séquence asynchrone de 0 à N événements, optionnellement terminée par un signal de fin ou une erreur.

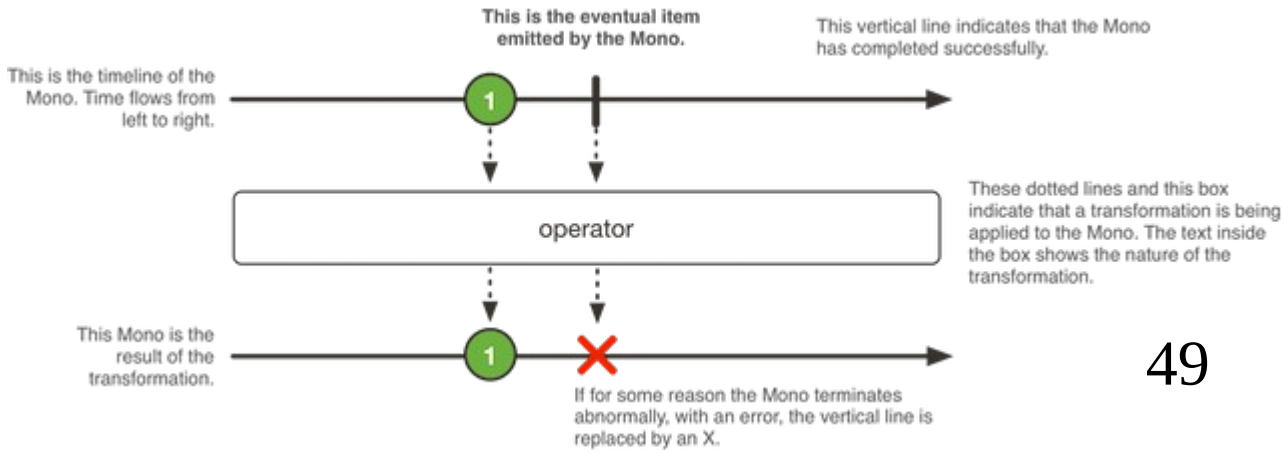
Les événements sont traduits par des appels de méthode sur les abonnés :

- Nouvelle valeur : *onNext()*
- Signal de fin : *onComplete()*
- Erreur : *onError()*

Mono

Mono<T> représente une séquence de 0 à 1 événements, optionnellement terminée par un signal de fin ou une erreur

Mono offre un sous-ensemble des opérateurs de Flux



Production d’un flux de données

La façon la plus simple de créer un *Mono* ou un *Flux* est d’utiliser les méthodes *Factory* à disposition.

```
Mono<Void> m1 = Mono.empty()
Mono<String> m2 = Mono.just("a");
Mono<Book> m3 = Mono.fromCallable(() -> new Book());
Mono<Book> m4 = mono.fromFuture(myCompletableFuture);


Flux<String> f1 = Flux.just("a","b","c");
Flux<Integer> f2 = Flux.range(0, 10);
Flux<Long> f3 = Flux.interval(Duration.ofMillis(1000).take(10);
Flux<String> f4 = Flux.fromIterable(bookCollection);
Flux<Book> f5 = Flux.fromStream(bookCollecion.stream());
```

Abonnement

L'abonnement au flux s'effectue via la méthode ***subscribe()***

Généralement, des lambda-expressions sont utilisées

```
subscribe();  // Déclenche le flux
subscribe(Consumer<? super T> consumer);
subscribe(Consumer<? super T> consumer,
          Consumer<? super Throwable> errorConsumer);
subscribe(Consumer<? super T> consumer,
          Consumer<? super Throwable> errorConsumer,
          Runnable completeConsumer);
// Chaînage
subscribe(Consumer<? super T> consumer,
          Consumer<? super Throwable> errorConsumer,
          Runnable completeConsumer,
          Consumer<? super Subscription> subscriptionConsumer);
```

Interface *Subscriber*

Sans utiliser les lambda-expression, on peut fournir une implémentation de l'interface ***Subscriber*** qui définit 4 méthodes :

```
void onComplete()
void onError(java.lang.Throwable t)
void onNext(T t)
void onSubscribe(Subscription s)
```

Invoqué après *Publisher.subscribe(Subscriber)*

Subscription

Subscription représente un abonnement d'un (seul) abonné à un *Publisher*.

Il est utilisé

- pour demander l'émission d'événement
  `void request(long n)`
- Pour annuler la demande et permettre la libération de ressource
  `void cancel()`

Exemple

```
Flux.just(1, 2, 3, 4)
  .log()
  .subscribe(new Subscriber<Integer>() {
    @Override
    public void onSubscribe(Subscription s) {
      s.request(Long.MAX_VALUE); // Provoque l'émission de tous les évts
    }

    @Override
    public void onNext(Integer integer) {
      elements.add(integer);
    }

    @Override
    public void onError(Throwable t) {}

    @Override
    public void onComplete() {}
});
```

Opérateurs

Les **opérateurs** permettent différents types d'opérations sur les éléments de la séquence :

- Transformer
- Choisir des événements
- Filtrer
- Gérer des erreurs
- Opérateurs temporels
- Séparer un flux
- Revenir au mode synchrone

Transformation

1 vers 1 :
map (nouvel objet), *cast* (chgt de type), *index*(Tuple avec ajout d'un indice)

1 vers N :
flatMap + une méthode factory, *handle*

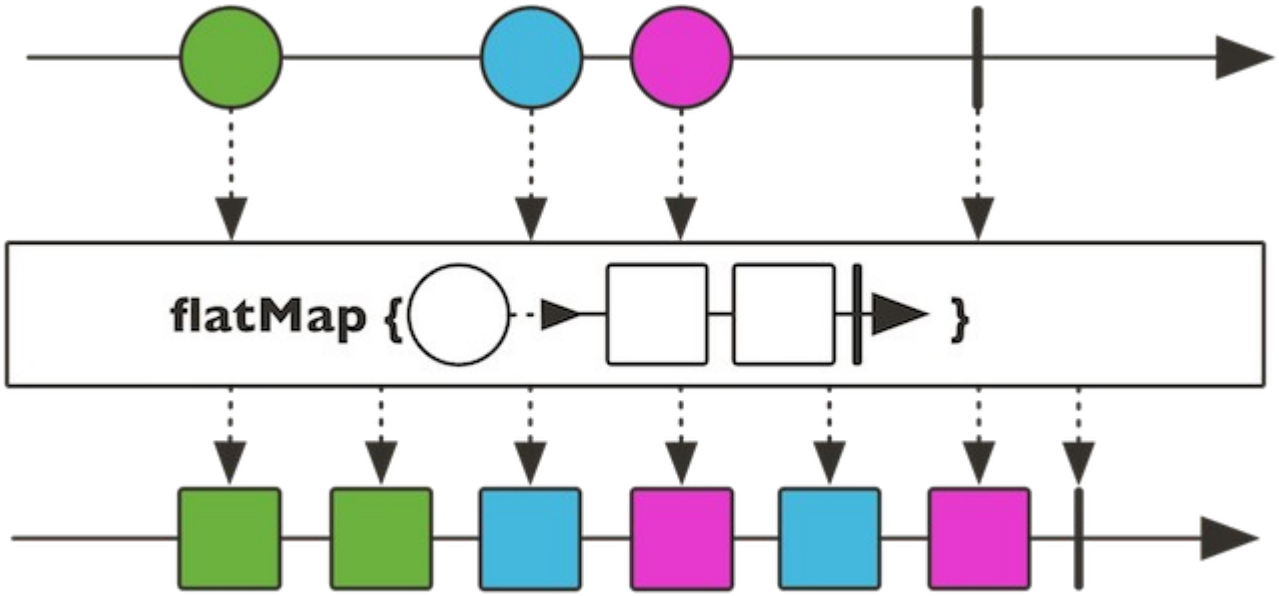
Ajouter des éléments à une séquence :
startsWith, *endWith*

Agréger :
collectList, *collectMap*, *count*, *reduce*, *scan*,

Agréger en booléen :
all, *any*, *hasElements*, *hasElement*

Combiner plusieurs flux :
concat, *merge*, *zip*

flatMap



Filtres

Filtre sur fonction arbitraire :
filter

Sur le type :
ofType

Ignorer toutes les valeurs :
ignoreElements

Ignorer les doublons :
distinct

Seulement un sous-ensemble :
take, ***takeLast***, ***elementAt***

Skipper des éléments :
skip(Long | Duration), skipWhile

Opérateurs temporels

Associé l'événement à un timestamp :
elapsed, timestamp

Séquence interrompue si délai trop important entre 2 événements :
timeout

Séquence à intervalle régulier :
interval

Ajouter des délais
Mono.delay, ***delayElements***, ***delaySubscription***

Threads, Scheduler

Introduction

Spring Reactor offre également du support pour contrôler la concurrence et l'utilisation des Threads

L'interface ***Scheduler*** permet de déléguer le traitement des événements à des threads séparés

Les opérateurs peuvent alors gérer des threads et des pools de workers.

Les objets *Flux*, *Mono* ou *Subscriber* n'ont pas de dépendance sur le modèle de concurrence.

Scheduler

L'interface ***Scheduler*** fournit une abstraction pour du code asynchrone

De nombreuses implémentations peuvent être obtenues à partir de méthodes factory de *Schedulers*

- ***fromExecutorService(ExecutorService)*** : Pool de threads à partir de *ExecutorService*.
- ***newParallel(java.lang.String)*** : Optimisé pour des exécutions rapides de *Runnable*
- ***single()*** : Optimisé pour des exécutions avec de faibles latences
- ***immediate()*** : Exécution immédiate des tâches sur la thread de l'appelant.

Opérateur et Scheduler

Certains opérateurs utilise par défaut une implémentation de *Scheduler*

Ils fournissent généralement la possibilité de spécifier une autre implémentation

Par exemple :

```
// Par défaut Schedulers.parallel()
Flux.interval(Duration.ofMillis(300))
// Positionnement de Schedulers.single()
Flux.interval(Duration.ofMillis(300),
  Schedulers.newSingle("test"))
```

Changer le contexte d'exécution

Reactor offre 2 moyen de changer le contexte d'exécution dans une chaîne réactive : ***publishOn*** et ***subscribeOn***.

La position de *publishOn* dans la chaîne a une importance pas celle de *subscribeOn*

- *publishOn* s'applique de la même façon que les autres opérateurs. Il prend le signal amont et le rejoue vers l'aval en exécutant le callback sur le worker du *Scheduler* associé.
- *subscribeOn* s'applique à l'abonnement lorsque la chaîne inverse est construite. Quelque soit sa position, il affecte tout le temps le contexte de la source de l'émission.
  Seul le subscribeOn le plus tôt dans la chaîne est pris en compte.

Gestion des erreurs, test et debugging

Introduction

Les erreurs sont des événements terminaux stoppant la séquence, arrêtant la chaîne d'opérateurs et provoquant l'appel de la méthode *onError()* de l'abonné.

Si *onError()* n'est pas définie, l'exception *UnsupportedOperationException* est lancée.

Opérateurs de traitement d’erreurs

Reactor permet de définir des opérateurs de traitement d’erreur en milieu de chaîne.

- Cependant, cela n‘évite pas l’arrêt de la séquence originale
- Cela permet de démarrer une nouvelle séquence : la séquence de *fallback*

Cas d’usage

Ces opérateurs permettent de retrouver les alternatives que l’on a avec les blocs *try, catch, finally* des *Exceptions*

1) Attraper et retourner une valeur statique par défaut
2) Attraper et exécuter une méthode de fallback.
3) Attraper et calculer une valeur de fallback.
4) Attraper, encapsuler dans une *BusinessException*, et relancer.
5) Attraper, log d’un message d’erreur et relancer.
6) Utiliser le bloc *finally* pour libérer les ressources ou "try-with-resource" de Java 7 .

Les opérateurs sont ***onErrorReturn, onErrorResume, doOnError, ...***

Exemples

```
// Valeur statique de fallback
Flux.just(10)
    .map(this::doSomethingDangerous)
    .onErrorReturn("RECOVERED");

// Méthode de fallback
Flux.just("key1", "key2")
    .flatMap(k -> callExternalService(k))
    .onErrorResume(e -> getFromCache(k)) );
    // (getFromCache retourne un Flux)
```

Exemples (2)

```
// Catch and rethrow
Flux.just("timeout1")
    .flatMap(k -> callExternalService(k))
    .onErrorResume(original -> Flux.error(
            new BusinessException("oops, SLA exceeded", original));
// Log (operator side-effect)
LongAdder failureStat = new LongAdder();
Flux<String> flux =
Flux.just("unknown")
    .flatMap(k -> callExternalService(k)
        .doOnError(e -> {
            failureStat.increment();
            log("uh oh, falling back, service failed for key " + k);
        })
        .onErrorResume(e -> getFromCache(k))
    );
```

Exemple (3)

```
// Finally
AtomicBoolean isDisposed = new AtomicBoolean();
Disposable disposableInstance = new Disposable() {
    @Override
    public void dispose() { isDisposed.set(true); }

    @Override
    public String toString() { return "DISPOSABLE"; }
};

Flux<String> flux =
Flux.using(
        () -> disposableInstance, // Génère la ressource
        disposable -> Flux.just(disposable.toString()),
        Disposable::dispose // <=> bloc finally
);
```

retry()

retry permet de se réabonner au *Publisher* pour lequel l'erreur s'est produite.

On obtient alors une nouvelle séquence, la séquence originale étant terminée

```
Flux.interval(Duration.ofMillis(250))
    .map(input -> {
        if (input < 3) return "tick " + input;
        throw new RuntimeException("boom");
    })
    .retry(1)
    .subscribe(System.out::println, System.err::println);
```

Quelle sortie ??

Exceptions dans les fonctions des opérateurs

Les exceptions lancées dans les fonctions des opérateurs peuvent être de 3 types :

- ***Fatal*** (*OutOfMemoryError*) : *Reactor* estime que l'on ne peut rien faire et lance l'exception
- ***Unchecked*** : Propagation vers la méthode onError
- ***Checked*** : Construction *try/catch* dans la fonction, avec possibilité d'utiliser une classe utilitaire *Exceptions*

Exemple

```
// Unchecked
Flux.just("foo")
    .map(s -> { throw new IllegalArgumentException(s); })
    .subscribe(v -> System.out.println("GOT VALUE"),
               e -> System.out.println("ERROR: " + e));


// Checked
Flux<String> converted = Flux
    .range(1, 10)
    .map(i -> {
        try { return convert(i); }
        catch (IOException e) { throw Exceptions.propagate(e); }
    });
```

Test

Reactor propose ***reactor-test***, un module pour les tests qui permet principalement 2 choses :

- Tester qu'une séquence suit un scénario donné avec ***StepVerifier.***
- Produire des données afin de tester le comportement d'opérateurs : ***TestPublisher***

```
// Gradle
dependencies {
   testcompile 'io.projectreactor:reactor-test'
}
```

StepVerifier

```
public <T> Flux<T> appendBoomError(Flux<T> source) {
  return source.concatWith(Mono.error(new IllegalArgumentException("boom")));
}
----------------------
@Test
public void testAppendBoomError() {
  Flux<String> source = Flux.just("foo", "bar");

  StepVerifier.create(
    appendBoomError(source))
    .expectNext("foo")
    .expectNext("bar")
    .expectErrorMessage("boom")
    .verify();
}
```

TestPublisher

```
TestPublisher<String> publisher = TestPublisher.create();
AtomicLong count = new AtomicLong();

Subscriber<String> subscriber = new CoreSubscriber<String>() {

  public void onError(Throwable t) { count.incrementAndGet(); }
  public void onComplete() { count.incrementAndGet(); }
};

publisher.subscribe(subscriber);
publisher.complete()
             .emit("A", "B", "C")
             .error(new IllegalStateException("boom"));

assertThat(count.get()).isEqualTo(1);
```

Debug

Dans le modèle impératif, le debug consiste principalement à lire la *stacktrace*

Cela devient un peu plus compliqué dans le modèle réactif, la stacktrace ne comporte souvent qu'une succession d'appels à *subscribe()* et *request()*

Reactor permet de fournir des stack-traces plus lisibles en activant le mode debug (impact sur les performances)

- Le mode debug peut être appliqué globalement ou à des points précis du code

Activation globale

L’activation globale s’effectue par :

```
Hooks.onOperatorDebug();
```

Cet appel permet d’instrumenter les appels aux opérateurs de *Flux* et *Mono*.

StackTrace

```
java.lang.IndexOutOfBoundsException: Source emitted more than one item
at reactor.core.publisher.MonoSingle$SingleSubscriber.onNext(MonoSingle.java:120)
at reactor.core.publisher.FluxOnAssembly$OnAssemblySubscriber.onNext(FluxOnAssembly.java:314)
...
...
at reactor.core.publisher.Mono.subscribeWith(Mono.java:2668)
at reactor.core.publisher.Mono.subscribe(Mono.java:2629)
at reactor.core.publisher.Mono.subscribe(Mono.java:2604)
at reactor.core.publisher.Mono.subscribe(Mono.java:2582)
at reactor.guide.GuideTests.debuggingActivated(GuideTests.java:727)
Suppressed: reactor.core.publisher.FluxOnAssembly$OnAssemblyException:
Assembly trace from producer [reactor.core.publisher.MonoSingle] :
reactor.core.publisher.Flux.single(Flux.java:5335)
reactor.guide.GuideTests.scatterAndGather(GuideTests.java:689)
reactor.guide.GuideTests.populateDebug(GuideTests.java:702)
org.junit.rules.TestWatcher$1.evaluate(TestWatcher.java:55)
org.junit.rules.RunRules.evaluate(RunRules.java:20)
Error has been observed by the following operator(s):
|_ Flux.single(TestWatcher.java:55)
```

Trace

L'opérateur ***log()*** permet de tracer les événements de la séquence

- Chaîné dans une séquence, il récupère tous les événements du stream amont (*onNext, onError, onComplete + subscriptions, cancellations et requests*).
- Il utilise la classe utilitaire *Loggers* qui retrouve le framework de logging (log4j, logback)

Exemple

```
Flux<Integer> flux = Flux.range(1, 10)
                         .log()
                         .take(3);
flux.subscribe()

-----
10:45:20.200 [main] INFO  reactor.Flux.Range.1 - |
   onSubscribe([Synchronous Fuseable] FluxRange.RangeSubscription)
10:45:20.205 [main] INFO  reactor.Flux.Range.1 - | request(unbounded)
10:45:20.205 [main] INFO  reactor.Flux.Range.1 - | onNext(1)
10:45:20.205 [main] INFO  reactor.Flux.Range.1 - | onNext(2)
10:45:20.205 [main] INFO  reactor.Flux.Range.1 - | onNext(3)
10:45:20.205 [main] INFO  reactor.Flux.Range.1 - | cancel()
```

Data et Web

Spring Data
Spring Webflux
WebClient
Reactive Web Sockets
Sécurité

Spring Data

Introduction

La programmation réactive s'invite également dans *SpringData*

Attention, cela ne concerne pas JDBC et JPA qui restent des APIs bloquantes

Sont supportés :

- MongoDB
- Cassandra
- Redis

Accès réactifs aux données persistante

Les appels sont asynchrones, non bloquants, pilotés par les événements

Les données sont traitées comme des flux

Cela nécessite :

- Spring Reactor
- Spring Framework 5
- Spring Data 2.0
- Un pilote réactif (Implémentation NoSQL exclusivement)
- Éventuellement Spring Boot (2.0)

Mélange bloquant non-bloquant

S’il faut mélanger du code bloquant et non bloquant, il ne faut pas bloquer la thread principale exécutant la boucle d’événements.

On peut alors utiliser les *Scheduler* de Spring Reactor.

Exemple :

```
Mono blockingWrapper = Mono.fromCallable(() -> {
    return /* make a remote synchronous call */
});
blockingWrapper = blockingWrapper.subscribeOn(Schedulers.elastic());
```

Apports

La fonctionnalité *Reactive* reste proche des concepts SpringData :

- API de gabarits réactifs (Reactive Templates comme *MongoReactiveTemplate*)
- Repository réactifs
- Les objets retournées sont des Flux ou Mono

Reactive Template

L'API des classes *Template* devient :

```
<T> Mono<T> insert(T objectToSave)
<T> Mono<T> insert(Mono<T> object)
<T> Flux<T> insertAll(Collection<? extends T> objectsToSave)
<T> Flux<T> insertAll(Flux<T> objectsToSave)
<T> Flux<T> find(Query query, Class<T> type
...
```

Exemple :

```
Flux<Person> insertAll = template
.insertAll(Flux.just(new Person("Walter", "White", 50), //
new Person("Skyler", "White", 45), //
new Person("Saul", "Goodman", 42), //
new Person("Jesse", "Pinkman", 27)).collectList());
```

Reactive Repository

L'interface ***ReactiveCrudRepository<T,ID>*** permet de profiter d'implémentations de fonction CRUD réactives.

Par exemple :

```
Mono<Long>  count()
Mono<Void>  delete(T entity)
Flux<T>   findAll()
Mono<S>   save(S entity)
..
```

Requêtes

Comme dans SpringData, les requêtes peuvent être déduites du nom des fonctions :

```
public interface ReactivePersonRepository extends
  ReactiveCrudRepository<Person, String> {

  Flux<Person> findByLastname(String lastname);

  @Query("{ 'firstname': ?0, 'lastname': ?1}")
  Mono<Person> findByFirstnameAndLastname(String firstname, String
  lastname);

  // Accept parameter inside a reactive type for deferred execution
  Flux<Person> findByLastname(Mono<String> lastname);

  Mono<Person> findByFirstnameAndLastname(Mono<String> firstname, String
  lastname);
}
```

Exemple dépendance *MongoDB* avec *SpringBoot*

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.0.0.BUILD-SNAPSHOT</version>
</parent>
< !-- Ramène en particulier : spring-data-mongodb et reactor-core -->
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>

   <artifactId>spring-boot-starter-data-mongodb-reactive</artifactId>
    </dependency>
</dependencies>
```

Spring Webflux

Motivation

2 principales motivations pour Spring Webflux :

- Le besoin d’un stack non-bloquante permettant de gérer la concurrence avec peu de threads et de scaler avec moins de ressources CPU/mémoire
- La programmation fonctionnelle

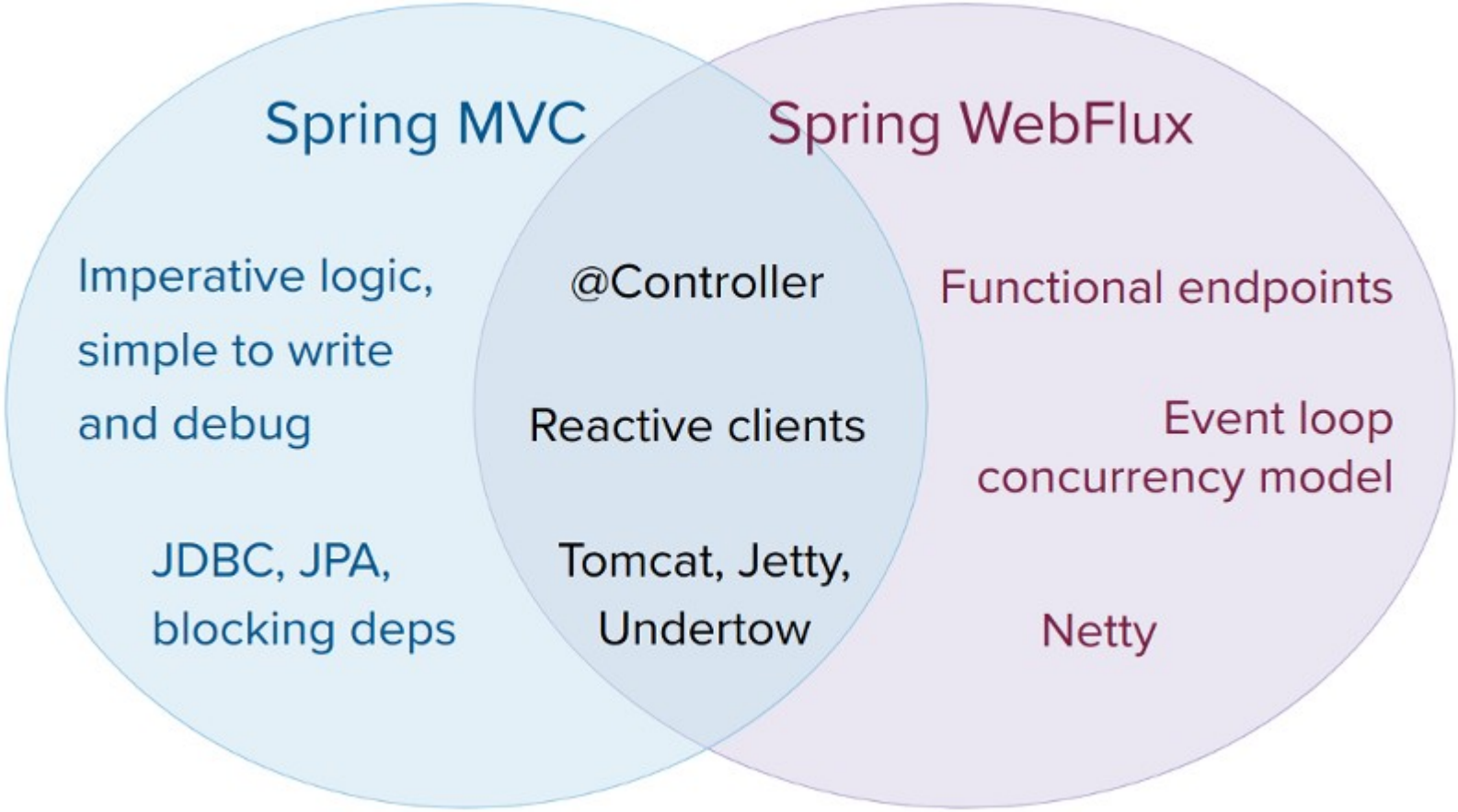
Introduction

2 modèles de programmation pour Spring Webflux :

- **Contrôleurs annotés** : Idem à Spring MVC avec les mêmes annotations.
  Les méthodes des contrôleurs peuvent retourner des types réactifs, des arguments réactifs sont associés à *@RequestBody*.

- **Endpoints fonctionnels** : Programmation fonctionnelle basée sur les lambdas.
  Idéal pour de petites libraires permettant de router et traiter des requêtes.
  Dans ce cas, le code applicatif est en charge du traitement de la requête du début à la fin.

MVC et WebFlux



Serveurs

Spring WebFlux est supporté sur

- Tomcat, Jetty, et les conteneurs de Servlet 3.1+,
- Ainsi que les environnements non-Servlet comme Netty ou Undertow

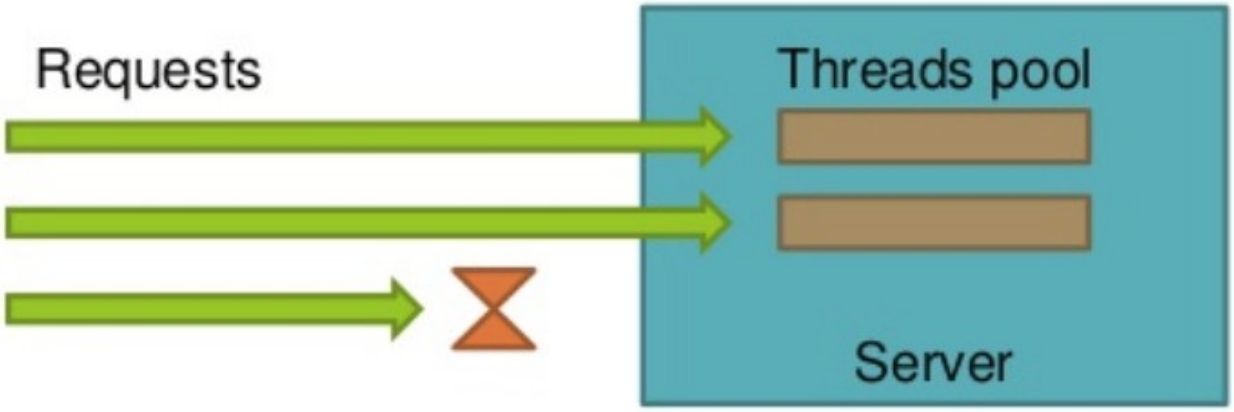
Le même modèle de programmation est supporté sur tous ces serveurs

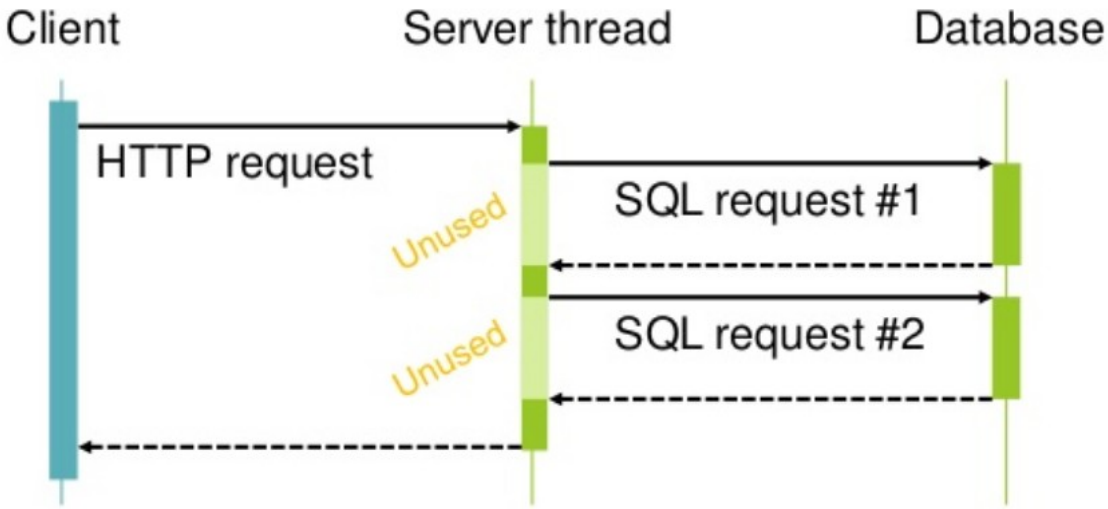
Avec *SpringBoot*, la configuration par défaut démarre un serveur embarqué *Netty*

Performance et Scaling

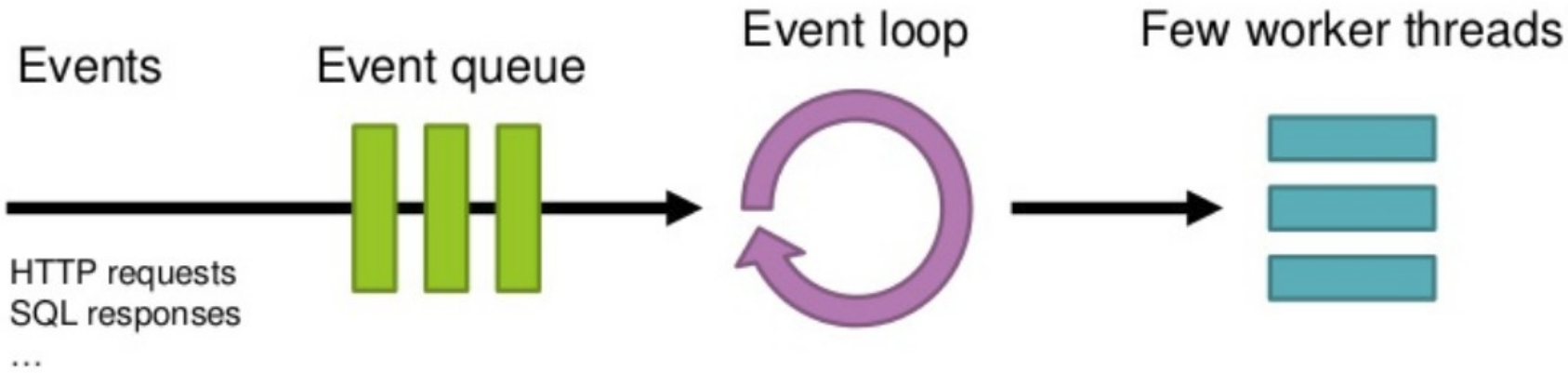
- Le modèle réactif/non bloquant n’apporte pas spécialement de gain en terme de temps de réponse. (cela peut même augmenter le temps de traitement)
- Le bénéfice attendu est la possibilité de **scaler** avec un petit nombre de threads fixes et moins de mémoire.
  - ==> Les applications sont plus résistantes à la charge.
- Idéal pour les architectures micro-services ou le traitement d’une requête nécessite plusieurs appels réseau avec de la latence.

Modèle bloquant





Modèle non bloquant



Modèle de threads

Pour un serveur *Spring WebFlux* entièrement réactif, on peut s'attendre à 1 thread pour le serveur et autant de threads que de CPU pour le traitement des requêtes.

Si on doit accéder à des données JPA ou JDBC par exemple, il est conseillé d'utiliser des *Schedulers* qui modifie alors le nombre de threads

Pour configurer le modèle de threads du serveur, il faut utiliser leur API de configuration spécifique ou voir si *Spring Boot* propose un support.

Généralités API

En général l'API WebFlux

- accepte en entrée un *Publisher*,
- l'adapte en interne aux types *Reactor*,
- l'utilise et retourne soit un *Flux,* soit un *Mono*.

En terme d'intégration :

- On peut fournir n'importe quel *Publisher* comme entrée
- Il faut adapter la sortie si l'on veut quelle soit compatible avec une autre librairie que *Reactor*

Contrôleurs annotés

Les annotations ***@Controller*** de Spring MVC sont donc supportés par *WebFlux*.

Les différences sont :

- Les beans cœur comme *HandlerMapping* ou *HandlerAdapter* sont non bloquants et travaillent sur les classes réactives
- ***ServerHttpRequest*** et ***ServerHttpResponse*** plutôt que *HttpServletRequest* et *HttpServletResponse*.

Exemple

```
@RestController
public class PersonController {
private final PersonRepository repository;
public PersonController(PersonRepository repository) {
  this.repository = repository;
}
@PostMapping("/person")
Mono<Void> create(@RequestBody Publisher<Person> personStream) {
  return this.repository.save(personStream).then();
}
@GetMapping("/person")
Flux<Person> list() {
  return this.repository.findAll();
}
@GetMapping("/person/{id}")
Mono<Person> findById(@PathVariable String id) {
  return this.repository.findOne(id);
}
}
```

Méthodes des contrôleurs

Les méthodes des contrôleurs ressemblent à ceux de Spring MVC (Annotations, arguments et valeur de retour possibles), à quelques exception prés

- Arguments :
  - ***ServerWebExchange*** : Encapsule, requête, réponse, session, attributs
  - ***ServerHttpRequest*** et ***ServerHttpResponse***
- Valeurs de retour :
  - ***Flux<ServerSentEvent>, Observable<ServerSentEvent>*** : Données + Méta-données
  - ***Flux<T>, Observable<T>*** : Données seules
- Request Mapping (consume/produce) : *text/event-stream*

Endpoints fonctionnels

Dans ce modèle de programmation fonctionnelle, les fonctions (lambda-expression) sont utilisées pour router et traiter les requêtes.

Les interfaces représentant l'interaction HTTP (requête/réponse) sont immuables
=> Thread-safe nécessaire pour le modèle réactif

ServerRequest et *ServerResponse*

ServerRequest et ***ServerResponse*** sont donc des interfaces qui offrent des accès via lambda-expression aux messages HTTP.

- ***ServerRequest*** expose le corps de la requête comme Flux ou Mono.
  Elle donne accès aux éléments HTTP (Méthode, URI, ..) à travers une interface séparée *ServerRequest.Headers*.
  ```
  Flux<Person> people = request.bodyToFlux(Person.class);
  ```
- ***ServerResponse*** accepte tout *Publisher* comme corps.
  Elle est créé via un builder permettant de positionner le statut, les entêtes et le corps de réponse
  ```
  ServerResponse.ok()
   .contentType(MediaType.APPLICATION_JSON).body(person);
  ```

Traitement des requêtes via *HandlerFunction*

Les requêtes HTTP sont traitées par une ***HandlerFunction*** : une fonction qui prend en entrée un *ServerRequest* et fournit un *Mono<ServerResponse>*

Exemple :

```
HandlerFunction<ServerResponse> helloWorld =
  request ->
  ServerResponse.ok().body(fromObject("Hello World"));
```

Généralement, les fonctions similaires sont regroupées dans une classe contrôleur.

Exemple

```
public class PersonHandler {

private final PersonRepository repository;

public PersonHandler(PersonRepository repository) { this.repository = repository;}

public Mono<ServerResponse> listPeople(ServerRequest request) {
  Flux<Person> people = repository.allPeople();
  return ServerResponse.ok().contentType(APPLICATION_JSON).body(people, Person.class);
}

public Mono<ServerResponse> createPerson(ServerRequest request) {
  Mono<Person> person = request.bodyToMono(Person.class);
  return ServerResponse.ok().build(repository.savePerson(person));
}

public Mono<ServerResponse> getPerson(ServerRequest request) {
int personId = Integer.valueOf(request.pathVariable("id"));
  Mono<ServerResponse> notFound = ServerResponse.notFound().build();
  Mono<Person> personMono = this.repository.getPerson(personId);
  return personMono
    .then(person -> ServerResponse.ok().contentType(APPLICATION_JSON).body(fromObject(person)))
    .otherwiseIfEmpty(notFound);
}
}
```

Mapping via *RouterFunction*

Les requêtes sont routées vers les *HandlerFunction* avec une ***RouterFunction :***

- Prend en entrée un *ServerRequest* et retourne un *Mono<HandlerFunction>*
- Les fonctions ne sont en général pas écrites directement. On utilise : ***RouterFunctions.route(RequestPredicate,HandlerFunction)*** permettant de spécifier les règles de matching

Exemple :

```
RouterFunction<ServerResponse> helloWorldRoute =
RouterFunctions.route(RequestPredicates.path("/hello-world"),
request -> Response.ok().body(fromObject("Hello World")));
```

Combinaison

2 fonctions de routage peuvent être composées en une nouvelle fonction via les méthodes

```
RouterFunction.and(RouterFunction)
RouterFunction.andRoute(RequestPredicate,
  HandlerFunction)
```

Si la première règle ne matche pas, la seconde est évaluée ... et ainsi de suite

Exemple

```
PersonRepository repository = ...
PersonHandler handler = new PersonHandler(repository);


RouterFunction<ServerResponse> personRoute = RouterFunctions.
  route(RequestPredicates.GET("/person/{id}")
    .and(accept(APPLICATION_JSON)), handler::getPerson)
  .andRoute(RequestPredicates.GET("/person")
    .and(accept(APPLICATION_JSON)), handler::listPeople)
  .andRoute(RequestPredicates.POST("/person")
    .and(contentType(APPLICATION_JSON)), handler::createPerson);
```

Exécution sur un serveur

Pour exécuter une *RouterFunction* sur un serveur, il faut le convertir en *HttpHandler*.

Différentes techniques sont possibles mais la + simple consiste à spécialiser une configuration ***WebFlux***

- La configuration par défaut apportée par ***@EnableWebFlux*** fait le nécessaire
- ou directement via SpringBoot et le starter *webflux*

Exemple

```
@Configuration
@EnableWebFlux
public class WebConfig implements WebFluxConfigurer {

    @Bean
    public RouterFunction<?> routerFunctionA() {  // … }

    @Bean
    public RouterFunction<?> routerFunctionB() { // … }

    @Override
    public void configureHttpMessageCodecs(ServerCodecConfigurer configurer) {
        // configure message conversion...
    }

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        // configure CORS...
    }

    @Override
    public void configureViewResolvers(ViewResolverRegistry registry) {
        // configure view resolution for HTML rendering...
    }
}
```

Filtres via *HandlerFilterFunction*

Les routes contrôlées par un fonction de routage peuvent être filtrées :

```
RouterFunction.filter(HandlerFilterFunction)
```

HandlerFilterFunction est une fonction prenant une *ServerRequest* et une *HandlerFunction* et retourne une *ServerResponse*.

Le paramètre *HandlerFunction* représente le prochain élément de la chaîne : la fonction de traitement ou la fonction de filtre.

Exemple :
Basic Security Filter

```
import static org.springframework.http.HttpStatus.UNAUTHORIZED;

SecurityManager securityManager = ...
RouterFunction<ServerResponse> route = ...

RouterFunction<ServerResponse> filteredRoute =
    route.filter((request, next) -> {
        if (securityManager.allowAccessTo(request.path())) {
            return next.handle(request);
        }
        else {
            return ServerResponse.status(UNAUTHORIZED).build();
        }
  });
```

Web Client

Introduction

WebFlux inclut ***WebClient*** : alternative non-bloquante à *RestTemplate*.

- Expose les I/O réseau via *ClientHttpRequest* et *ClientHttpResponse*.
  - Les corps de la requête et de la réponse sont des *Flux<DataBuffer>* plutôt que des *InputStream* et *OutputStream*.
- Même mécanismes de sérialisation (JSON, XML) permettant de travailler avec des objets typés.
- En interne, *WebClient* délègue à une librairie client HTTP (Par défaut, *Reactor Netty*)

Création

La façon la + simple de créer un *WebClient* est d'utiliser les méthodes statiques :

```
WebClient.create()
WebClient.create(String baseUrl)
```

On obtient alors un *HttpClient* de *Reactor Netty* avec les configurations par défaut

Il existe également un *WebClient.Builder* qui permet de préciser toutes les options de configuration

Une fois construit, une instance de *WebClient* est immuable.

Réponse

La méthode ***retrieve()*** permet de récupérer une réponse et de la décoder :

```
WebClient client = WebClient.create("http://example.org");

Mono<Person> result = client.get()
                            .uri("/persons/{id}", id)
                            .accept(MediaType.APPLICATION_JSON)
                            .retrieve()
                            .bodyToMono(Person.class);
```

Exceptions

- Les réponses avec des statuts *4xx* ou *5xx* provoquent une ***WebClientResponseException***.
- Il est possible d'utiliser la méthode ***onStatus*** pour personnaliser le traitement de l'exception:

```
Mono<Person> result = client.get()
            .uri("/persons/{id}", id).accept(MediaType.APPLICATION_JSON)
            .retrieve()
            .onStatus(HttpStatus::is4xxServerError, response -> ...)
            .onStatus(HttpStatus::is5xxServerError, response -> ...)
            .bodyToMono(Person.class);
```

Contrôle de la réponse

La méthode ***exchange()*** permet un meilleur contrôle de la réponse en permettant d'avoir un accès à *ClientResponse*

```
Mono<Person> result = client.get()
            .uri("/persons/{id}", id).accept(MediaType.APPLICATION_JSON)
            .exchange()
            .flatMap(response -> response.bodyToMono(Person.class));
```

Corps de requête

Le corps de la requête peut être encodé à partir d'un *Mono*, d'un *Flux* ou d'une type simple:

```
Mono<Person> personMono = ... ;

Mono<Void> result = client.post()
          .uri("/persons/{id}", id)
          .contentType(MediaType.APPLICATION_JSON)
          .body(personMono, Person.class)
          .retrieve()
          .bodyToMono(Void.class);
```

Corps de la requête (2)

```
Flux<Person> personFlux = ... ;

Mono<Void> result = client.post()
          .uri("/persons/{id}", id)
          .contentType(MediaType.APPLICATION_STREAM_JSON)
          .body(personFlux, Person.class)
          .retrieve()
          .bodyToMono(Void.class);

Person person = ... ;

  Mono<Void> result = client.post()
          .uri("/persons/{id}", id)
          .contentType(MediaType.APPLICATION_JSON)
          .syncBody(person)
          .retrieve()
          .bodyToMono(Void.class);
```

Formulaire

Pour poster des données de formulaire, il faut fournir une *MultiValueMap<String, String>* dans le corps.

Le content type est alors positionné automatiquement à : "*application/x-www-form-urlencoded*"

```
MultiValueMap<String, String> formData = ... ;


Mono<Void> result = client.post()
          .uri("/path", id)
          .syncBody(formData)
          .retrieve()
          .bodyToMono(Void.class);
```

Filtre client

Le ***WebClient.Builder*** permet d'enregistrer un filtre qui intercepte les requêtes

Exemple : Authentification Basique

```
WebClient client = WebClient.builder()
        .filter(basicAuthentication("user", "password"))
        .build();
```

Reactive Web Sockets

Introduction

Le protocole ***WebSocket*** (RFC 6455) définit un standard pour une communication full-duplex entre un client et un serveur.

C’est un protocole TCP différent de HTTP mais qui utilise les ports 80 et 443 pour passer les firewall.

Spring fournit une API *WebSocket* permettant d’écrire du code client ou serveur.

Cas d’usage

Une combinaison d’Ajax et de streaming HTTP ou du polling peuvent souvent avoir les mêmes effets

Les *WebSockets* sont utilisées lorsque le client et le serveur doivent échanger des événements à une haute fréquence avec peu de latence.

Attention, la configuration de beaucoup de proxy bloque les websockets !!

URL unique

A la différence des architectures HTTP ou REST, les *WebSockets* n'utilisent qu'**une seule URL** pour la connexion initiale

Tous les messages applicatifs utilisent alors la même connexion TCP.

=> Cela conduit à une architecture asynchrone et piloté par les événements

Mise en place

La mise en place consiste à :

- Définir côté serveur un ***WebSocketHandler***
- L'associer à une URL via un ***HandlerMapping*** et un ***WebSocketHandlerAdapter***
- Utiliser un ***WebSocketClient*** pour démarrer un session

WebSocketHandler

WebSocketHandler définit la méthode *handle()* qui prend une *WebSocketSession* et retourne *Mono<Void>* lorsque le traitement de la session est terminée

La session est traitée via 2 streams de WebSocketMessage : 1 pour le message d'entrée, 1 pour le message de sortie

La session propose donc 2 méthodes :

- ***Flux<WebSocketMessage> receive()*** : Accès au flux d'entrée se termine à la fermeture de connexion.
- ***Mono<Void> send(Publisher<WebSocketMessage>)*** : Prend un source pour les messages de sortie, écrit les messages et retourne *Mono<Void>* lorsque la source est tarie.

Exemple

```
@Component
public class ReactiveWebSocketHandler implements WebSocketHandler {

    // private fields ...

    @Override
    public Mono<Void> handle(WebSocketSession webSocketSession) {
        return webSocketSession.send(intervalFlux
          .map(webSocketSession::textMessage))
          .and(webSocketSession.receive()
            .map(WebSocketMessage::getPayloadAsText)
            .log());
    }
}
```

Mapping

```
@Autowired
private WebSocketHandler webSocketHandler;

@Bean
public HandlerMapping webSocketHandlerMapping() {
    Map<String, WebSocketHandler> map = new HashMap<>();
    map.put("/event-emitter", webSocketHandler);

    SimpleUrlHandlerMapping handlerMapping = new SimpleUrlHandlerMapping();
    handlerMapping.setOrder(1);
    handlerMapping.setUrlMap(map);
    return handlerMapping;
}

@Bean
public WebSocketHandlerAdapter handlerAdapter() {
   return new WebSocketHandlerAdapter();
}
```

Client

Spring WebFlux fournit une interface ***WebSocketClient*** avec des implémentations pour Reactor Netty, Tomcat, Jetty, Undertow, et Java standard (i.e. JSR-356).

Pour démarrer une session *WebSocket*, créer une instance du client et utiliser sa méthode ***execute*** :

```
WebSocketClient client = new ReactorNettyWebSocketClient();

URI url = new URI("ws://localhost:8080/path");
client.execute(url, session ->
        session.receive()
                .doOnNext(System.out::println)
                .then());
```

Spring Security 5.x

Introduction

Spring Security apporte les nouveautés suivantes :

- Meilleur support de *OAuth 2.0*
- Support pour la programmation réactive
  - *@EnableWebFluxSecurity*
  - *@EnableReactiveMethodSecurity*
  - *ReactiveUserDetailsService*
  - Test de WebFlux
- Nouveaux encodages de mots de passe

OAuth 2.0

OAuth 2.0 permet aux utilisateurs de se connecter sur un application en se connectant avec un compte existant d'un fournisseur oAuth2.0

- Se logger avec son compte GitHub ou Google
- Protéger des micro-services dans une architecture micro-services

Oauth2.0 est un protocole avec bcp de variantes d'implémentations

SpringBoot / Authentification Google

SpringBoot permet de facilement mettre en place une authentification Google par exemple :

- Obtenir un *clientId* et un *clientSecret* chez Google
- Positionner l'URL de redirection permettant à Google de fournir le jeton d'autorisation
- Configurer *application.yml* en indiquant les crédentiels client
- Se connecter à l'application et consentir que celle-ci accède à l'adresse email google et les informations basiques de profil

WebFlux Security

La sécurité pour *Webflux* est consistante avec celle de Spring MVC

- Elle est implémentée sous forme de **filtre** (WebFilter) que l'on peut configurer finement
- L'annotation ***@EnableWebFluxSecurity*** permet d'avoir une configuration par défaut.
- La configuration par défaut peut être personnalisé via la classe ***ServerHttpSecurity***

Les Beans de personnalisation sont réactifs

Configuration minimale

```
/* Configuration fournissant un authentification basique
* via une page de login et de logout
* Toutes les URLs sont protégées
* Les entêtes HTTP relatifs à la sécurité sont positionnés (CSRF, …)
*/
@EnableWebFluxSecurity
public class HelloWebfluxSecurityConfig {

/*
* MapReactiveUserDetailsService implémente ReactiveUserDetailsService
*/
@Bean
public MapReactiveUserDetailsService userDetailsService() {
UserDetails user = User.withDefaultPasswordEncoder()
  .username("user")
  .password("user")
  .roles("USER")
  .build();
return new MapReactiveUserDetailsService(user);
}
}
```

Configuration personnalisée

```
@EnableWebFluxSecurity
public class HelloWebfluxSecurityConfig {


@Bean
public SecurityWebFilterChain
  springSecurityFilterChain(ServerHttpSecurity http) {
  http
     .authorizeExchange()
     .anyExchange().authenticated()
     .and()
     .httpBasic().and()
     .formLogin();
     return http.build();
  }
}
```

Sécurité au niveau des méthodes

En programmation réactive, la sécurité au niveau méthodes est possible en utilisant des Reactor's context

Elle peut être cumulée avec la sécurité sur les URLS

De la même façon, on obtient une configuration par défaut en utilisant l'annotation ***@EnableReactiveMethodSecurity***

Exemple minimal

```
@EnableReactiveMethodSecurity
public class SecurityConfig {
@Bean
public MapReactiveUserDetailsService userDetailsService() {
  User.UserBuilder userBuilder = User.withDefaultPasswordEncoder();
  UserDetails rob =
 userBuilder.username("rob").password("rob").roles("USER").build();
  UserDetails admin =
 userBuilder.username("admin").password("admin").roles("USER","ADMIN").build();
  return new MapReactiveUserDetailsService(rob, admin);
}
}
----------------------------------
@Component
public class HelloWorldMessageService {

  @PreAuthorize("hasRole('ADMIN')")
  public Mono<String> findMessage() {
    return Mono.just("Hello World!");
  }
}
```

Test de la sécurité

- Spring Security 5 offre un support pour tester la sécurité pour tout type de combinaison : Impératif/Réactif et URL/méthodes
- Annotations : *@WithMockUser, @WithAnonymousUser, @WithUserDetails, MockMVC*
- Dans l'univers réactif : Utilisation de ***StepVerifier*** provenant de *Reactor* permettant d'exprimer les événements attendus d'un *Publisher* lors d'un abonnement

Spring Coeur et les tests

Spring Coeur
Annotations Tests
WebTestClient

Framework cœur

Grâce aux amélioration de la réflexion de Java 8, les paramètres des méthodes peuvent être efficacement accédées.

Les interfaces cœur de Spring proposent des méthodes par défaut (Java 8).

Les annotations *@Nullable* et *@NotNull* permettent de marquer explicitement les arguments et valeurs de retour pouvant être *null*.
=> Permet de déplacer les *NullPointerException* à la compilation.

Spring 5.0 embarque « *Commons Logging bridge* » (*spring-jcl*). Cela permet de détecter Log4j 2.x, SLF4J, JUL (*java.util.logging*) sans effort.

Conteneur

Il est possible de définir un index de composants dans ***META-INF/spring.components***.

- A la compilation, le source est introspecté et les entités JPA et les composants Spring sont marqués.
- Cette fonctionnalité accélère le temps de boot de Spring

L'annotation ***@Nullable*** peut être utilisée pour des injections optionnels. Cela impose que le consommateur se prépare à traiter le *null*

Programmation fonctionnel pour *GenericApplicationContext* et *AnnotationConfigApplicationContext* => Beans fonctionnels

Les espaces de nom pour la configuration XML n'utilisent plus de version de schémas.

Kotlin

```
// Exemple Web
{
("/movie" and accept(TEXT_HTML)).nest {
GET("/", movieHandler::findAllView)
GET("/{card}", movieHandler::findOneView)
}
("/api/movie" and accept(APPLICATION_JSON)).nest {
GET("/", movieApiHandler::findAll)
GET("/{id}", movieApiHandler::findOne)
}
}
// Exemple enregistrement de bean
val context = GenericApplicationContext {
registerBean()
registerBean { Cinema(it.getBean()) }
}
```

Beans fonctionnels

La classe *GenericApplicationContext* propose une nouvelle version de *registerBean()* recevant 2 interfaces fonctionnelles comme paramètres :

- *Supplier* pour créer l'objet
- *BeanDefinitionCustomizer* : permettant de fournir une ou plusieurs lambda pour personnaliser *BeanDefinition*

Exemple

```
// 1 lambda pour créer le bean
// 1 lambda pour le personnaliser
context.registerBean(
  "myCallbackService",
  MyService.class,
   () -> new MyService(),
  bd -> bd.setAutowireCandidate(false));
```

Annotations tests

JUnit5

Spring 5 ajoute de nouvelles annotations compatibles exclusivement avec JUnit5

- ***@SpringJUnitConfig*** et ***@SpringJUnitWebConfig*** facilitant la création du contexte Spring
- ***@EnabledIf/@DisabledIf*** : Permettant de activer/désactiver des tests selon des conditions

@SpringJUnitConfig

@SpringJUnitConfig combine 2 annotations:

- ***@ExtendWith(SpringExtension.class)*** de JUnit 5 pour exécuter les tests avec l'extension *SpringExtension*
- ***@ContextConfiguration*** de Spring Testing pour charger le contexte Spring

Exemple

```
@SpringJUnitConfig(SpringJUnitConfigIntegrationTest.Config.class)
public class SpringJUnitConfigIntegrationTest {

    @Configuration
    static class Config {}


    @Autowired
    private ApplicationContext applicationContext;

    @Test
    void givenAppContext_WhenInjected_ThenItShouldNotBeNull() {
      assertNotNull(applicationContext);
    }
}
```

@SpringJUnitWebConfig

@SpringJUnitWebConfig est une combinaison de *@SpringJUnitConfig* + *@WebAppConfiguration*

Il permet de charger un *WebApplicationContext*.

```
@SpringJUnitWebConfig(TestConfig.class)
class ConfigurationClassJUnitJupiterSpringWebTests {
    // class body...
}
```

Activer/Désactiver des tests

Les annotations ***@EnabledIf*** et ***@DisabledIf*** permettent d'activer ou désactiver des tests selon une expression :

- Spring Expression Language (SpEL)

```
@EnabledIf("#{systemProperties['os.name'].contains('Mac')}")
```

- Référence à une propriété de l'environnement Spring

```
@EnabledIf("${smoke.tests.enabled}")
```

- Texte

```
@EnabledIf("true")
```

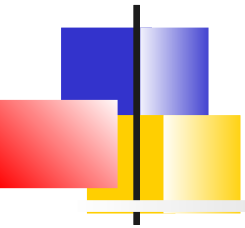
WebTest Client

Support pour Webflux

Spring5 propose des mock implémentations de *ServerHttpRequest* , *ServerHttpResponse*, et *ServerWebExchange* à utiliser dans des applications WebFlux.

Le ***WebTestClient*** permet de tester une application WebFlux

- Sans serveur : Test unitaire
- Avec serveur : Test e2e



Mise en place

La création d'un ***WebTestClient*** s'effectue de différentes façons.

En fonction du type de test que l'on veut effectuer on peut l'associer à :

- Un unique contrôleur
- Une *RouterFunction*
- Un contexte Spring
- Un serveur

Contrôleur unique

```
client = WebTestClient.bindToController(new
  TestController()).build();
```

- Charge la configuration de WebFlux
- Enregistre le contrôleur fournit .

L’application WebFlux peut être testée sans serveur HTTP en utilisant des requêtes et des réponses mockées.

D’autres méthodes sur le builder permettent de personnaliser la config.

RouterFunction

```
RouterFunction<?> route = ...

    client =
  WebTestClient.bindToRouterFunction(route).build();
```

L’application *WebFlux* peut également être testée sans serveur HTTP en utilisant des requêtes et des réponses mockées.

ApplicationContext

```
@RunWith(SpringRunner.class)
@ContextConfiguration(classes = WebConfig.class)
public class MyTests {
        @Autowired
        private ApplicationContext context;

        private WebTestClient client;

        @Before
        public void setUp() {
            client =
  WebTestClient.bindToApplicationContext(context).build();
        }
    }
```

L'application *WebFlux* peut également être testée sans serveur HTTP en utilisant des requêtes et des réponses mockées.

Serveur

```
client = WebTestClient.bindToServer().
     baseUrl("http://localhost:8080").build();
```

=> Connexion à un serveur démarré

Client Builder

- On peut configurer les options du client comme la base URL, les entêtes par défaut, les filtres, ..
- Ces options sont directement accessible lorsque l'on a associé le client à un serveur, pour les autres association, il faut utiliser ***configureClient()***

```
client = WebTestClient.bindToController(new
  TestController())
            .configureClient()
            .baseUrl("/test")
            .build();
```

Ecriture des tests

- *WebTestClient* fournit une API identique à *WebClient*
- L'exécution d'une requête peut se faire par ***exchange()***
- Ensuite, une chaîne de vérification peut être combinée.
  - Tester le statut et et les entêtes
  - Extraire le corps de la réponse et y effectuer des assertions

Exemple

```
client.get().uri("/persons/1")
            .accept(MediaType.APPLICATION_JSON_UTF8)
            .exchange()
           .expectStatus().isOk()
           .expectHeader()
             .contentType(MediaType.APPLICATION_JSON_UTF8)
           .expectBodyList(Person.class) // Extraction
             .hasSize(3).contains(person); // Assertions
```

Extraction du corps de requête

Typiquement, 3 possibilités pour extraire le corps de requête :

- ***expectBody(Class<T>)*** : Décoder en un simple objet.
- ***expectBodyList(Class<T>)*** : Décoder en une List<T>.
- ***expectBody()*** : Décoder en un tableau d'octets pour le contenu JSON ou un corps vide

Le résultat effectif peut être récupéré

Exemple avec obtention du résultat

```
EntityExchangeResult<Person> result = client.get().uri("/persons/1")
            .exchange()
            .expectStatus().isOk()
            .expectBody(Person.class)
            .returnResult();
```

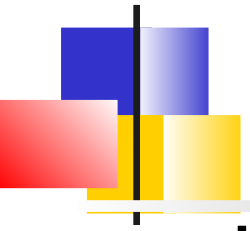
Réponse sans contenu

```
// Permet de garantir que les ressources sont libérées
client.get().uri("/persons/123")
            .exchange()
            .expectStatus().isNotFound()
            .expectBody(Void.class);
// Assertion pour contenu vide
client.post().uri("/persons")
            .body(personMono, Person.class)
            .exchange()
            .expectStatus().isCreated()
            .expectBody().isEmpty();
```

Réponse JSON

Possibilité d'utiliser *JSONAssert* et *JSONPath*

```
client.get().uri("/persons")
            .exchange()
            .expectStatus().isOk()
            .expectBody()
            .jsonPath("$[0].name").isEqualTo("Jane")
            .jsonPath("$[1].name").isEqualTo("Jason");
```



Réponses Stream

Le test des réponses sous forme de flux, s'effectue en 2 étapes :

- Récupérer un Flux via la méthode *returnResult*
- Utiliser StepVerifier de Reactor pour vérifier les événements du Flux.

Exemple

```
FluxExchangeResult<MyEvent> result = client.get().uri("/events")
            .accept(TEXT_EVENT_STREAM)
            .exchange()
            .expectStatus().isOk()
            .returnResult(MyEvent.class);


Flux<Event> eventFux = result.getResponseBody();


    StepVerifier.create(eventFlux)
            .expectNext(person)
            .expectNextCount(4)
            .consumeNextWith(p -> ...)
            .thenCancel()
            .verify();
```

Merci!!!

- MERCI DE VOTRE ATTENTION

Références

Spring Reference : Web Reactive Stack :
https://docs.spring.io/spring/docs/5.1.0.RC3/spring-framework-reference/web-reactive.html#spring-webflux

Spring Reactor :
http://projectreactor.io/docs/core/release/reference

Tutoriaux Baeldung

https://www.baeldung.com/spring-5

Présentations

https://fr.slideshare.net/fbeaufume/programmation-ractive-avec-spring-5-et-reactor-81924228

https://fr.slideshare.net/Pivotal/reactive-data-access-with-spring-data